



Chaos Engineering Framework for Microservices Resilience Testing

CSE6207

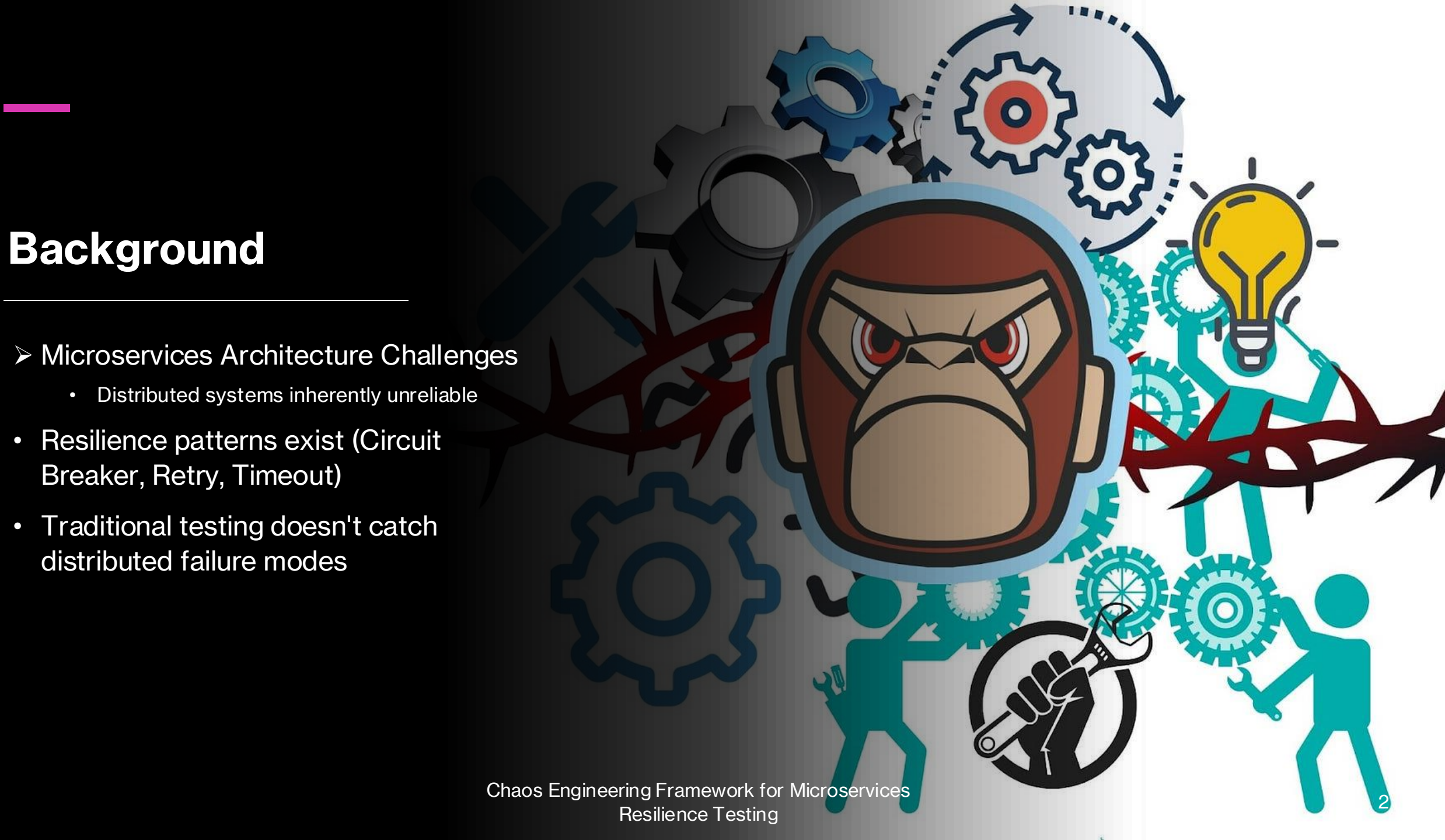
S.M.A Sayem Prodhan

ID: 0424052101



Background

- Microservices Architecture Challenges
 - Distributed systems inherently unreliable
- Resilience patterns exist (Circuit Breaker, Retry, Timeout)
- Traditional testing doesn't catch distributed failure modes



Problem Statement

- Validation Gap
 - Unknown behavior under real failure conditions
- Testing Limitations
 - Unit tests don't capture distributed failures
- Lack of Systematic Approach
 - Ad-hoc failure injection

Research Questions

How can we systematically validate and quantify the effectiveness of resilience patterns in microservices architectures?

Project Aim

- Design and implement a **chaos engineering framework** to systematically test microservices resilience patterns
 - Build a microservices application with 5+ services implementing standard resilience patterns
 - Develop an **automated fault injection framework** supporting multiple failure types
 - Conduct controlled experiments measuring system behavior under failures
 - Analyze effectiveness of resilience patterns through quantitative metrics

Motivation – SOA Gap

Existing Approaches:

Approach	Limitations
Netflix Chaos Monkey	Production-only, no controlled experiments
Jepsen	Database-focused, complex to setup
Gremlin	Commercial, black-box testing

Proposed Solution

Two-Component System:

1. Chaos Engineering Framework
2. Test Microservices Application

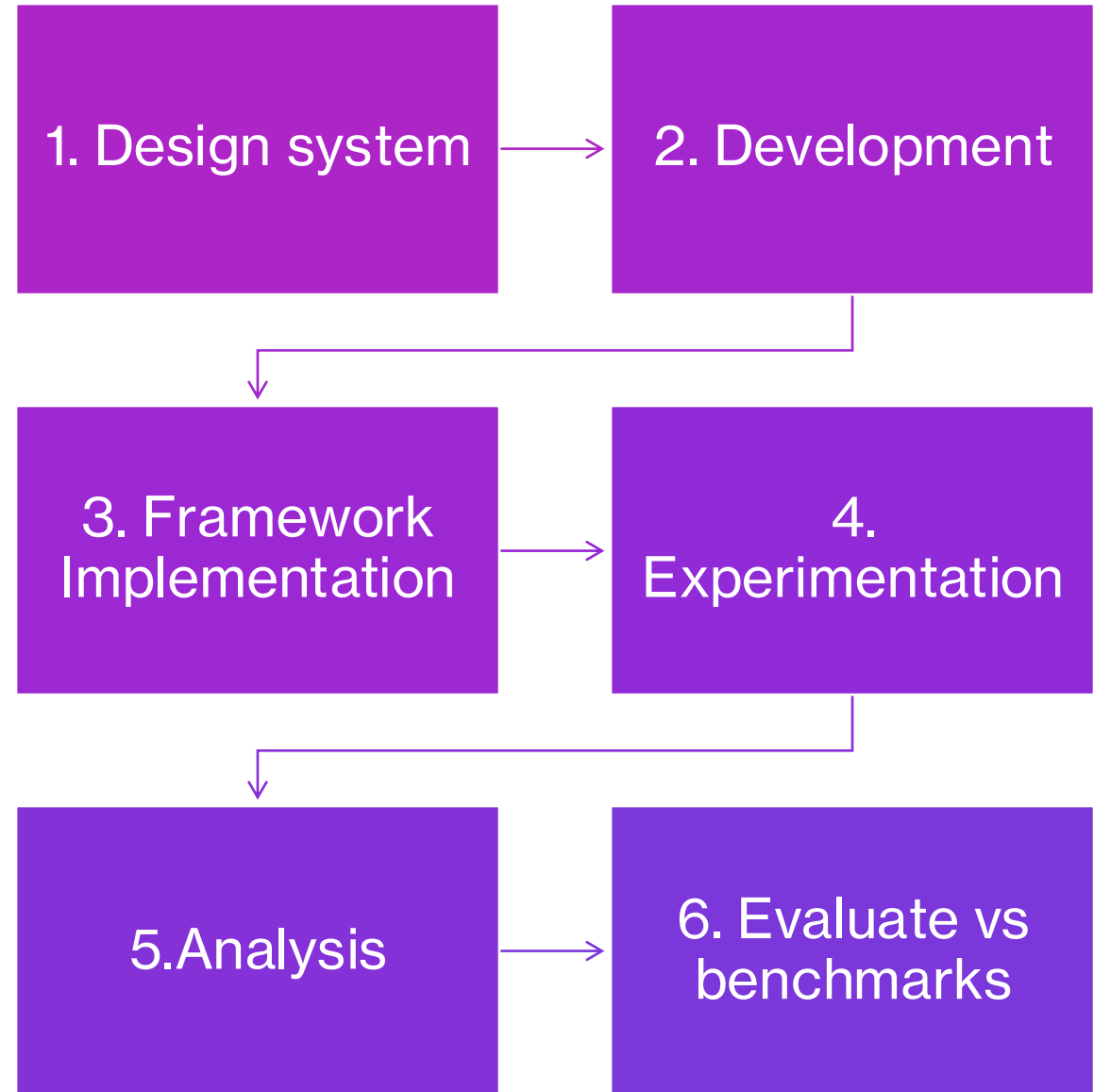
Proposed Solution

- Chaos Engineering Framework
 - Fault Injector (6 types)
 - Experiment Orchestrator
 - Metrics Collector
 - Analysis Engine
- Fault Types:
 - Latency
 - Service Crash
 - Error Rate
 - Network Partition
 - Resource Exhaustion
 - Packet Loss

Proposed Solution

- Testing
 - 5 services (simulated E-commerce)
 - 6 resilience patterns
 - Configurable fault tolerance
- Resilience Patterns:
 - Circuit Breaker
 - Retry with Backoff
 - Timeout
 - Bulkhead
 - Fallback
 - Health Checks

Methodology



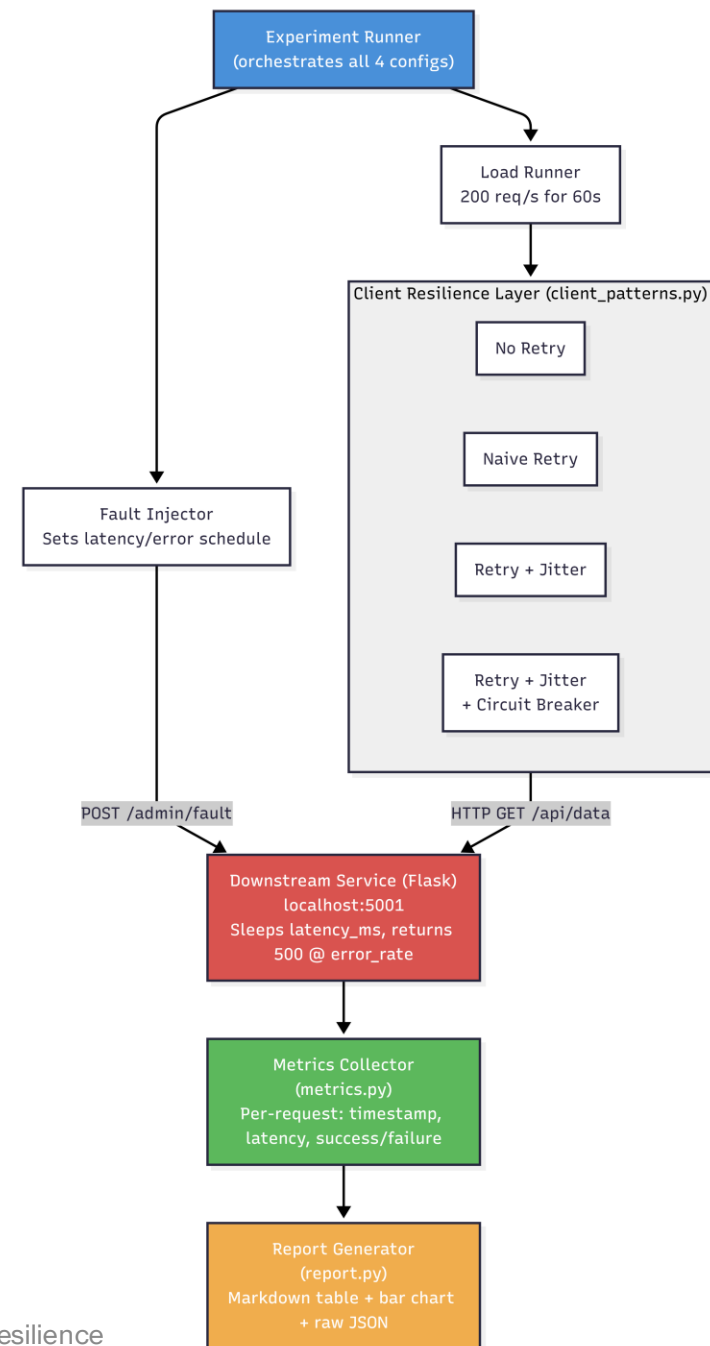
Expected Outcomes

- Working Systems
 - Chaos engineering framework
 - Experiment automation scripts
- Better system compared to the literature review

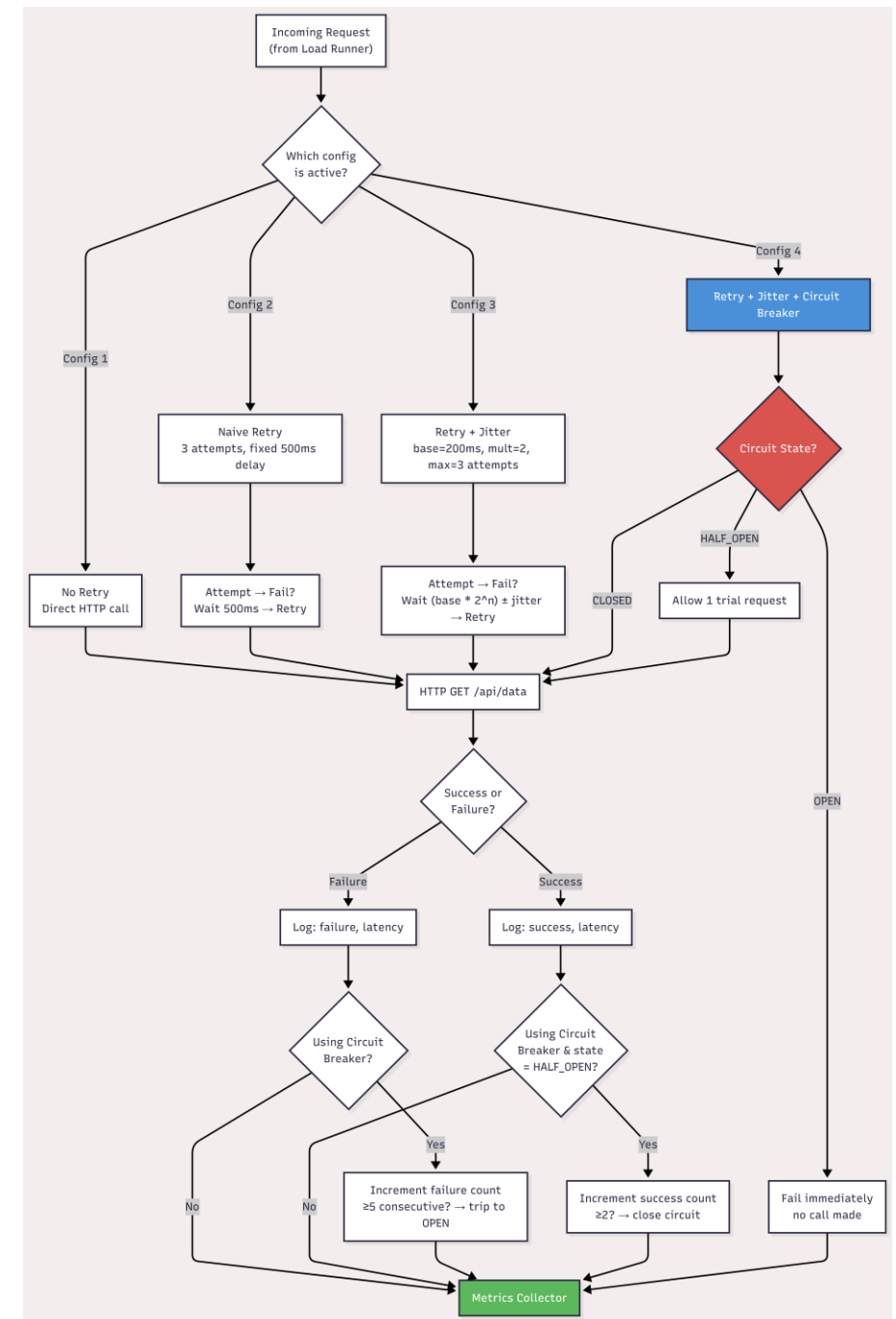
Target Benchmark

Metric	Literature Baseline	Literature Best-Case	Source
Error rate under retry storm	17%	3% (bounded retry + circuit breaker)	Mohammad (2025), SLR
P99 latency under retry storm	2,600 ms	1,100 ms	Mohammad (2025), SLR
System availability	97.80%	99.96%	Oduri, S. (2025)
Microservices failure rate	43.4% (Sock Shop)	-	Chen, Goudarzi & Toosi (2025), arXiv

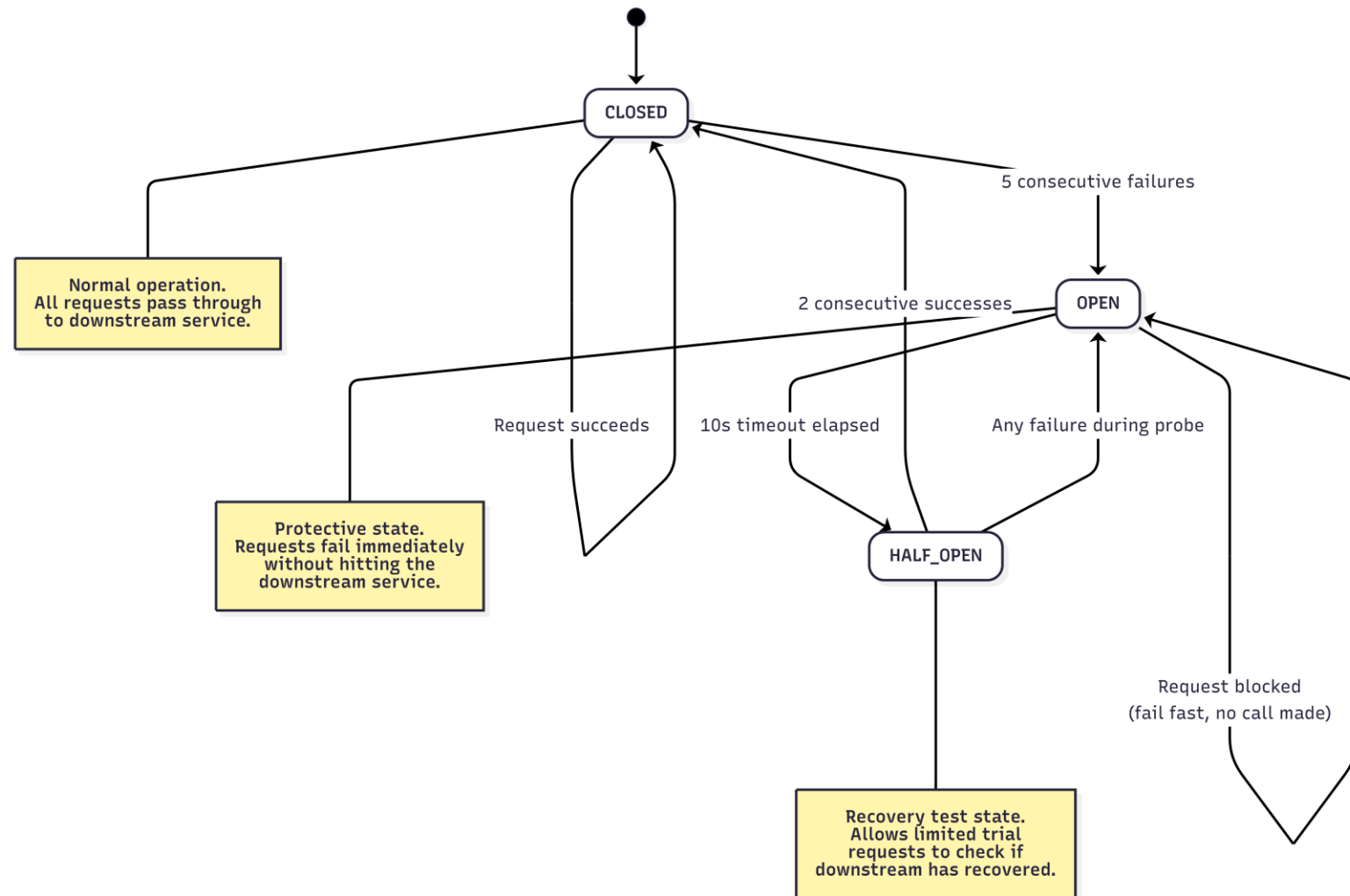
System Design



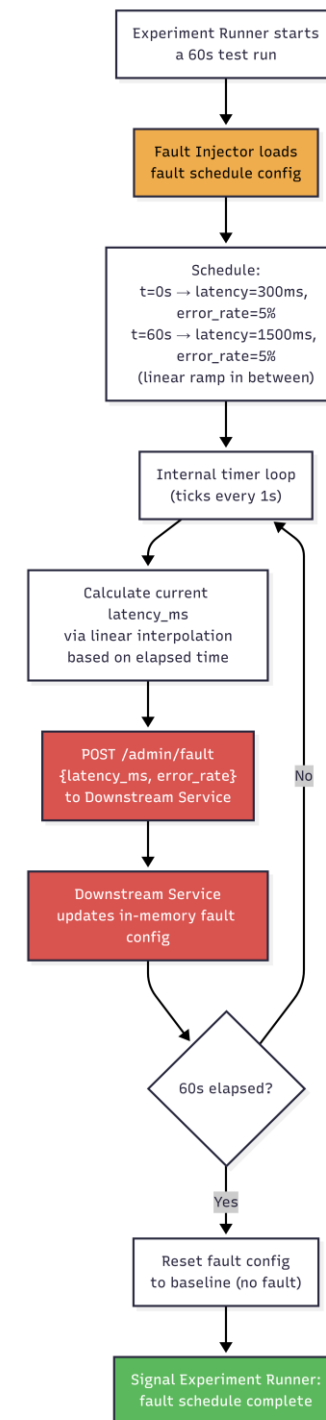
Load runner



The Circuit Breaker



The Fault Injector



References

1. Mohammad, M. (2025). Resilient Microservices: A Systematic Review of Recovery Patterns, Strategies, and Evaluation Frameworks. arXiv:2512.16959.
2. Oduri, S. (2025). Engineering Resilience: Cloud-Native Design Patterns for Fault-Tolerant Systems. Vol. 32, No. 2 (2025).
3. Chen, Z., Goudarzi, M., & Toosi, A. N. (2025). Resilience Evaluation of Kubernetes in Cloud-Edge Environments via Failure Injection. arXiv:2507.16109
4. Basiri, A. et al. (2016). "Chaos Engineering." IEEE Software. Netflix Tech Blog.
5. Nygard, M. (2018). Release It! 2nd Edition. Pragmatic Bookshelf. [Circuit Breaker, Bulkhead patterns]
6. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly. [Distributed systems fault tolerance]
7. Humble, J. & Farley, D. (2010). Continuous Delivery. Addison-Wesley. [Testing strategies for distributed systems]
8. Alvaro, P. et al. (2015). "Lineage-driven Fault Injection." SIGMOD. [Systematic fault injection methods]
9. Netflix Tech Blog - Chaos Engineering series (2011-present)



Thank you